

Protocol Audit Report

MortgageFi – cbBTC Earn Pool

Version 1.0

wafardev

November 5, 2026

Lead Security Researcher: wafardev
Scope: `mortgagefiusdccbbtcupgraded.sol` (`0xE931...2DC7`)
Target Network: Base Mainnet

Protocol Audit Report

Prepared by: wafardev
Lead Security Researcher: wafardev
Date: November 5, 2026

Contents

1 Protocol Summary **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Audit Details **4**
4.1 Scope 4
4.2 Roles 4

5 Executive Summary **4**

6 Findings **5**
6.1 Critical 5

---

1 Protocol Summary

MortgageFi is a decentralized non-custodial mortgage and lending protocol that enables users to purchase tokenized assets (e.g. cbBTC, WETH) with down payments while liquidity providers (LPs) earn yield by depositing stablecoins (USDC) into Earn pools.

The **cbBTC Earn Pool** (`0xE931...2DC7`) on Base enables depositors to supply USDC backing for cbBTC mortgages. Depositors acquire LP shares that entitle them to stream interest repaid by mortgage borrowers. Under normal operations, depositors can exit by entering a withdrawal queue to redeem their underlying USDC.

2 Disclaimer

Wafardev makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

3 Risk Classification

We utilize the standard severity matrix evaluating Impact versus Likelihood:

Impact	High Likelihood	Medium Likelihood	Low Likelihood
High	Critical (C)	High (H)	Medium (M)
Medium	High (H)	Medium (M)	Low (L)
Low	Medium (M)	Low (L)	Informational (I)

4 Audit Details

4.1 Scope

The following contract was reviewed as part of the audit:

- **Target Contract**: `mortgagefiusdccbbtcupgraded.sol`
- **Deployed Address**: `0xE93131620945 A1273b48F57f453983d270b62DC7`
- **Network**: Base Mainnet
- **Pinned Fork Block**: `53533000`
- **Solc Version**: `^0.8.4`
- **Total Value Locked (TVL)**: ~$989,872 USD (9.733 cbBTC + 335,558 USDC liabilities; 947,675 LP shares)

4.2 Roles

- **Admin**: Protocol administrator account holding `ADMIN_ROLE` and `DEFAULT_ADMIN_ROLE`: `0x87d2E27147A6 40092e06c9329bF8110ca9927446`
- **Depositors (LPs)**: Permissionless actors who supply USDC into `deposit(receiver, amount)` to earn mortgage yield and redeem via the withdrawal queue.
- **Borrowers**: Users who open collateralized mortgage positions via `createContract(...)` or `createContractByob(...)`.

5 Executive Summary

A targeted security assessment and forked state machine audit was performed against MortgageFi's cbBTC Earn Pool on Base. Bytecode analysis, runtime call-trace verification, and Foundry simulation were employed.

One **Critical** severity vulnerability was identified and formally verified. Depositor exit mechanics are broken due to an erroneous administrative modifier attached to the user exit function, leading to a permanent on-chain capital freeze for all liquidity providers.

Severity	Number of Issues Found
Critical	1
High	0
Medium	0
Low	0
Informational	0
Total	**1**

6 Findings

6.1 Critical

[C-01] Erroneous `onlyRole(ADMIN_ROLE)` on `burnToGetOutInLine` Permanently Traps Depositor Capital

Severity: Critical
Impact: High (Complete loss of withdrawal capabilities / permanent freeze of $989k TVL)
Likelihood: High (Affects 100% of liquidity provider redemptions)
Target Contract: `mortgagefiusdccbbtcupgraded.sol` (`0xE931...2DC7`)

Description: MortgageFi implements a two-stage withdrawal mechanism for Earn pool liquidity providers:

1. **Phase 1 (`burnToGetOutInLine`)**: Depositors burn their LP shares to enter the withdrawal queue, incrementing their queued balance (`mySizeWaitingForExit[msg.sender]`) and total queue size (`inExitLine`).

2. **Phase 2 (`fromLineToWallet`)**: As repayments or unborrowed reserves become available, depositors claim their stablecoins from the queue via `stablecoin.safeTransfer(msg.sender, size)`.

In the deployed cbBTC pool on Base (`0xE931...2DC7`), the entry function `burnToGetOutInLine` was erroneously compiled and deployed with the `onlyRole(ADMIN_ROLE)` modifier:

```
    function burnToGetOutInLine(uint256 amount)
        giveYield()
        fetch(msg.sender)
        onlyRole(ADMIN_ROLE) // @audit Erroneous administrative access control
        external
    {
        // get in line to exit the protocol
        // burn shares
        _burn(msg.sender, amount);
        // add to size
        mySizeWaitingForExit[msg.sender] += amount;
        inExitLine += amount;
    }

```

Because `burnToGetOutInLine` takes no user address argument and strictly operates on `msg.sender`:

- Any regular user attempting to withdraw has their transaction reverted with `AccessControl: account is missing role ADMIN_ROLE`.
- Users cannot populate `mySizeWaitingForExit[msg.sender]`. Consequently, direct calls to Phase 2 (`fromLineToWallet`) revert with an arithmetic underflow (`0 - size`).
- The Admin cannot call `burnToGetOutInLine` on behalf of users, as the function explicitly burns `msg.sender`'s shares and credits `msg.sender`'s exit balance.

In contrast, in the WETH pool on Base (`0x1bE8...9EEA`), `burnToGetOutInLine` is completely permissionless. This confirms that the modifier in the cbBTC pool was mistakenly added during parameter setter refactoring.

Impact: All depositor capital placed into the cbBTC Earn pool is permanently locked on-chain and cannot be redeemed through normal protocol operations. Sovereign depositor entitlement is broken for ∼$989,872 USD of backing assets.

Proof of Concept: The following Foundry test reproduces the vulnerability against pinned Base block 53533000:

```
    function test_Vulnerability_UserFundsPermanentlyLocked_cbBTC_Pool() public {
        address alice = makeAddr("alice");
        address admin = 0x87d2E27147A640092e06c9329bF8110ca9927446;
        uint256 depositAmount = 10_000 * 1e6; // 10,000 USDC

        deal(address(usdc), alice, depositAmount);

        // 1. Alice deposits 10,000 USDC into the pool permissionlessly
        vm.startPrank(alice);
        usdc.approve(address(pool), type(uint256).max);
        pool.deposit(alice, depositAmount);

        uint256 aliceShares = pool.balanceOf(alice);
        assertEq(aliceShares, depositAmount, "Alice receives 1:1 LP shares");

        // 2. Alice attempts to withdraw via burnToGetOutInLine
        // Reverts with AccessControl error due to erroneous ADMIN_ROLE modifier
        vm.expectRevert(
            "AccessControl: account 0x328809bc894f92807417d2dad6b7c998c1afdac6 "
            "is missing role 0xdf8b4c520ffe197c5343c6f5aec59570151ef9a492f2c624fd45ddde6135ec42"
        );
        pool.burnToGetOutInLine(aliceShares);

        // 3. Direct call to Phase 2 (fromLineToWallet) reverts with underflow
        vm.expectRevert();
        pool.fromLineToWallet(depositAmount);
        vm.stopPrank();

        // 4. Admin cannot process Alice's withdrawal (only burns admin's own shares)
        vm.startPrank(admin);
        uint256 adminBefore = pool.balanceOf(admin);
        pool.burnToGetOutInLine(100 * 1e6);
        assertEq(pool.balanceOf(admin), adminBefore - 100 * 1e6);
        assertEq(pool.balanceOf(alice), aliceShares, "Alice shares remain trapped");
        vm.stopPrank();
    }

```

Execute the test with:

```
forge test --match-test test_Vulnerability_UserFundsPermanentlyLocked_cbBTC_Pool -vv
```

Terminal test execution result:

```
[PASS] test_Vulnerability_UserFundsPermanentlyLocked_cbBTC_Pool() (gas: 1307895)
Logs:
  1. Alice successfully deposited 10,000 USDC and received shares: 10000000000
  2. Alice's burnToGetOutInLine reverted: missing ADMIN_ROLE
  3. Alice's fromLineToWallet reverted: underflow because queue is blocked
  4. Admin can only burn admin's own shares; cannot process Alice's withdrawal
  5. Verified: WETH pool allows permissionless user exit; cbBTC pool is broken
```

Recommended Mitigation:

1. **Source Code Patch**: Remove the `onlyRole(ADMIN_ROLE)` modifier from `burnToGetOutInLine`:

```
- function burnToGetOutInLine(uint256 amount) giveYield() fetch(msg.sender)
-     onlyRole(ADMIN_ROLE) external {
+ function burnToGetOutInLine(uint256 amount) giveYield() fetch(msg.sender)
+     external {
        _burn(msg.sender, amount);
        mySizeWaitingForExit[msg.sender] += amount;
        inExitLine += amount;
    }

```

2. **On-Chain Remediation for Deployed State**: Because the deployed contract is non-upgradeable:

   - The Admin account holds `DEFAULT_ADMIN_ROLE`: `0x87d2E271 47A640092e06c9329bF8110ca9927446`
   - Deploy a permissionless **Exit Router Adapter** contract and grant it `ADMIN_ROLE` via `grantRole(ADMIN_ROLE, routerAddress)`.
   - The Router contract can accept user LP shares, hold `ADMIN_ROLE`, invoke `burnToGetOutInLine`, and forward USDC returns directly to depositors.
   - Alternatively, deploy a patched pool contract and provide a 1:1 liquidity migration bridge.